

İSTANBUL TOPKAPI UNIVERSITY
FACULTY OF ENGINEERING AND NATURAL SCIENCES
DEPARTMENT OF COMPUTER ENGINEERING



Computer Architecture and Organization

Project Name: Programmable Counter

Digital Design(25 points)	Verilog Implementation (25 points)	Verilog Testbench (25 points)	Academic Report and Presentation (25 points)	Total (100 points)

	Student Name and Surname	Student ID	Signature
1	Miraç Aydın	24040102028	
2	Salih Artuç	23040102018	
3	Samiullah Arman	24040102002	
4	Yusuf Eren Şerbetçi	23040102024	

GitHub Repository :

<https://github.com/YusufErenSerbetci/programmable-counter-verilog.git>

1. Introduction

Digital counters are fundamental components in many digital systems such as processors, timers, and embedded controllers. They are used to track events, generate sequences, and control timing operations.

In this project, a 4-bit programmable counter is designed using Verilog Hardware Description Language. The counter supports three main operations: counting up, counting down, and loading a predefined value. The design is fully synchronous and operates with respect to a clock signal, ensuring stable and predictable behavior.

2. Project Description

The aim of this project is to design and implement a programmable counter that can operate in multiple modes. The system allows the user to dynamically switch between counting directions and also load a custom value directly into the counter.

The design includes the following control signals:

Clock: Synchronizes all operations

Reset: Clears the counter to zero

Enable: Activates the counter

Load: Loads an external value

UpOrDown: Determines counting direction

The output of the counter is a 4-bit value that updates on each clock cycle.

3. Digital Design Explanation

The circuit diagram (shown in Figure 1) illustrates the internal structure of the counter. The system is built using four D-type flip-flops, each representing one bit of the output.

Each flip-flop has a multiplexer at its input. This allows the circuit to either:

- Load a value from external inputs (b0–b3), or
- Continue counting based on logic operations

Combinational logic, including XOR and AND gates, is used to determine the next state of each flip-flop. The upOrDown signal controls whether the counter increments or decrements.

- When upOrDown = 0, the counter counts up
- When upOrDown = 1, the counter counts down

The Reset signal has the highest priority and forces the counter to zero regardless of other signals.

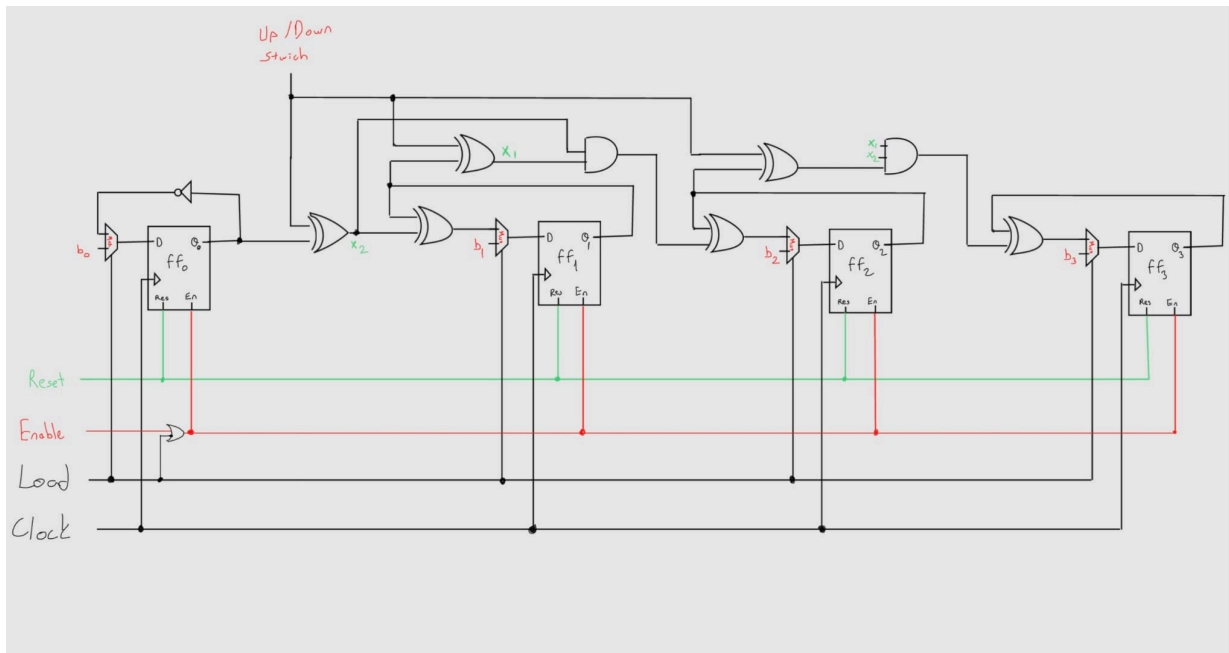


Figure 1: Circuit Diagram

4. Design Details and Operation

The counter is a synchronous design, meaning all flip-flops are triggered at the same clock edge. This approach improves performance compared to asynchronous (ripple) counters.

The behavior of the counter follows a priority order:

- Reset (highest priority)
- Load
- Counting
- Up/Down

If the Enable signal is low, the counter holds its current value and does not change.

This design ensures:

- Stable operation
- Predictable outputs
- Easy control of the counter

5. Verilog Implementation and Verification

The programmable counter is implemented using a behavioral Verilog model. The design is based on a synchronous structure, where all operations are controlled by the rising edge of the clock signal.

The core functionality is defined inside an always block that is sensitive to both the clock and reset signals. When the Reset signal is active, the counter output is immediately cleared to zero. This ensures that the system always starts from a known state.

When the Enable signal is active, the counter performs its main operations. If the Load signal is asserted, the input value is directly transferred to the output, allowing the counter to start from any desired value. If the Load signal is not active, the counter operates in either up-counting or down-counting mode depending on the value of the upOrDown signal. When upOrDown is high, the counter increments its value by one at each clock cycle. When it is low, the counter decrements its value.

The implementation follows a clear priority structure. Reset has the highest priority, followed by Load, and finally the counting operation. This ensures predictable and stable behavior of the system.

To verify the correctness of the design, a testbench is created. The testbench generates a clock signal using a periodic toggle mechanism and applies different input scenarios to test all functionalities of the counter.

During simulation, the following cases are tested:

- Reset operation to confirm proper initialization
- Up counting behavior
- Down counting behavior
- Parallel load functionality

The testbench also includes waveform dumping using \$dumpfile and \$dumpvars, which allows the simulation results to be visualized in GTKWave.

The simulation results confirm that the counter behaves correctly under all tested conditions. The output transitions match the expected values, and all control signals function as intended. This demonstrates that both the Verilog implementation and the verification process are successful.

counter.v

```
module counterBasis (  
    input Clock,  
    input Reset,  
    input Enable,  
    input Load,  
    input upOrDown,  
    input [3:0] in,  
    output reg [3:0] out  
);  
always @(posedge Clock or posedge  
Reset) begin  
    if (Reset) begin  
        out <= 4'b0000;  
    end  
    else if (Enable) begin  
        if (Load) begin  
            out <= in;  
        end  
        else if (upOrDown) begin  
            out <= out + 1'b1;  
        end  
        else begin  
            out <= out - 1'b1;  
        end  
    end  
end  
endmodule
```

testbench.v

```
`timescale 1ns/1ps  
module tb_counter;  
    reg Clock;  
    reg Reset;  
    reg Enable;  
    reg Load;  
    reg upOrDown;  
    reg [3:0] in;  
    wire [3:0] out;  
    counterBasis asd (  
        .Clock(Clock),  
        .Reset(Reset),  
        .Enable(Enable),  
        .Load(Load),  
        .upOrDown(upOrDown),  
        .in(in),  
        .out(out)  
    );  
    initial begin  
        Clock = 0;  
        forever #5 Clock = ~Clock;  
    end  
    initial begin  
        $dumpfile("testbench.vcd");  
        $dumpvars(0, tb_counter);  
        Reset = 1; Enable = 0; Load = 0;  
        upOrDown = 1; in = 4'b0000;  
        #15 Reset = 0; Enable = 1;  
        #30;  
        Reset = 1;  
        #10;  
        Reset = 0;  
        #50;  
        upOrDown = 0;  
        #40;  
        in = 4'b1010;  
        Load = 1;  
        #10 Load = 0;  
        upOrDown = 1;  
        #80;  
        $finish;  
    end  
endmodule
```

Figure 2 : Verilog Code

6. Simulation Results

The simulation results are obtained using GTKWave (in Figure 3).

Observations:

- When Reset = 1 , output becomes 0000
- When Enable = 1 , counter starts operating
- Counting direction changes correctly
- Load operation works instantly

The waveform confirms that the design behaves correctly under all tested conditions.

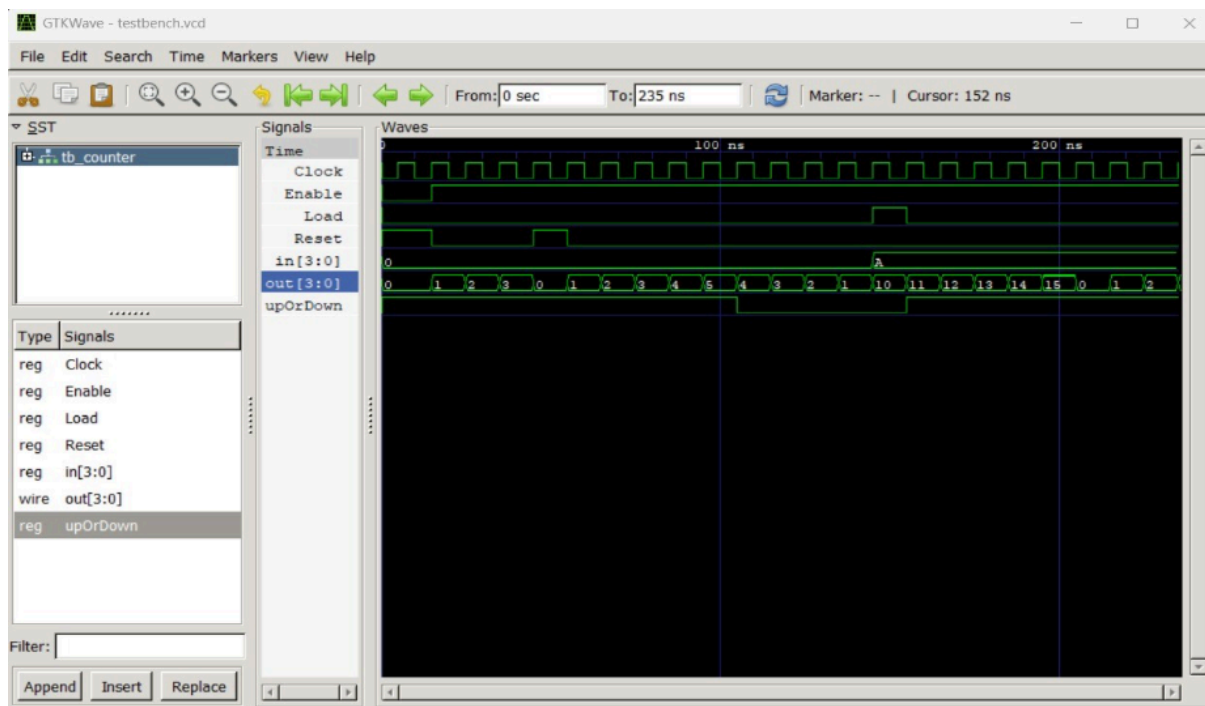


Figure 3: GTKWave Output

7. Key Features of the Design

This design has several important features:

- Ability to load any 4-bit value instantly
- Dynamic switching between up and down counting
- Fully synchronous operation
- Controlled by Enable signal

These features make the counter more flexible than a simple binary counter.

8. Conclusion

The simulation results confirm that the system works correctly and meets all design requirements.

This project helped us gain a better understanding of synchronous digital design and Verilog-based hardware modeling.